



Implementation Guide for Adoption of FHIR in ABDM and NHCX

Created by:
National Resource Centre for EHR Standards,
Centre for Development of Advanced Computing (C-DAC), Pune, India

Published: September 2024

TABLE OF CONTENTS

Introduction	4
Brief Introduction to FHIR	4
Scope	4
Key Concepts of FHIR	5
• Resource:	5
• Bundle	5
• Profile and Extension:	5
• Terminology	5
• Validation	5
• Interoperability:	5
• Modularity and Scalability:	5
• Exchange Paradigm:	5
Resource	6
Key Part of Resource	7
Data types	7
Key datatypes:	7
FHIR Paradigm	8
Bundle	9
Use of Bundle with 'type' as 'Document' and 'Collection'	9
Use of Bundle with 'type' as 'Document'	9
How it works:	9
Example Use Case:	9
Use of Bundle with 'type' as 'Collection'	9
Example Use Case:	9
Key Elements of a FHIR Bundle resource	10
FHIR Implementation Guide	10
FHIR Implementation Guide for Ayushman Bharat Digital Mission	11
Reading FHIR Profiles	11
1. Statistics/References	11
2. Differential View	11
3. Mandatory Element	11
4. Must Support 	12
FHIR Profiles for ABDM	12
FHIR Profiles for NHCX	13
Implementing and Validating FHIR Programmatically in Java	14

HAPI FHIR Library	14
Key Features of HAPI FHIR Library	14
• Complete FHIR Support:	14
• Built-in Validators:	14
• Serialization & Parsing:	14
• Extensive Resource Coverage:	14
HAPI FHIR Dependencies	14
• hapi-fhir-structures-r4:	14
• hapi-fhir-validation:	14
• hapi-fhir-validation-resources-r4:	14
Creating FHIR Resources Programmatically	15
Prerequisites	15
Setting Up the Development Environment	15
Creating Resource	16
Validating FHIR Resources	16
Validation Tools	16
Validation Aspects	17
Validation Process	17
Implementation Reference	20

INTRODUCTION

Intending to build a national digital health ecosystem that provides diverse data and infrastructure services by leveraging open, interoperable systems, adopting FHIR in Ayushman Bharat Digital Mission (ABDM) has been a cornerstone of ABDM's digital healthcare initiatives.

FHIR has been identified as a data structure standard defining health information structures that represent different health records to achieve continuity of care along with standard structures defined for claim processing. The adoption of FHIR in ABDM and NHCX results in enhanced healthcare delivery and fosters innovation in health services. It streamlines data exchange between different health systems, leading to better interoperability and more accurate patient records. It also facilitates real-time access to health information, which can support more timely and informed decision-making by healthcare providers. FHIR's standardized protocols streamline claims processing and administrative workflows leading to faster claim processing and reduced errors.

BRIEF INTRODUCTION TO FHIR

Fast Healthcare Interoperability Resources (FHIR), developed by Health Level Seven (HL7), is a modern standard designed to streamline the electronic exchange of healthcare information. By using widely adopted web standards such as RESTful APIs, XML, and JSON, FHIR provides a flexible framework that simplifies healthcare data integration and ensures seamless interoperability between different healthcare systems.

One of the primary goals of FHIR is to simplify healthcare data exchange by reducing technical barriers. Using familiar internet-based technologies, it allows real-time sharing of healthcare information through discrete “Resources” such as patient data, medications, and observations. These modular resources can be easily combined and extended to suit various healthcare needs, supporting both data sharing and the development of new applications. FHIR's standardized framework promotes interoperability across systems, enabling faster innovation and more accurate clinical decision-making, ultimately improving patient outcomes and healthcare delivery.

Refer: [Index - FHIR v4.0.1 \(hl7.org\)](https://hl7.org/fhir/index.html)

SCOPE

This document provides an overview of FHIR and its usage within the ABDM. It serves as a reference for understanding FHIR and its adoption in ABDM and NHCX, outlining how to create and validate FHIR resources using available libraries. While the approach to implementation may vary based on application requirements, resources, and scope, this document aims to assist healthcare

stakeholders in integrating interoperable, secure, and standardized healthcare data exchange solutions effectively within the healthcare ecosystem.

KEY CONCEPTS OF FHIR

- **Resource:** FHIR is resource-centric, meaning all healthcare-related data is represented as a set of modular components called "resource." These resources are the building blocks of FHIR and can represent anything from a patient, medication, or observation, to complex care plans. Resources can be combined or extended to suit specific use cases, making FHIR adaptable to various needs.
- **Bundle:** FHIR supports the use of Bundle, which is collections of resources that can be sent or retrieved in a single transaction.
- **Profile and Extension:** FHIR resources can be customized using profiles to meet specific implementation needs. Extensions allow adding new data elements or modifying existing ones without altering the core resource structure.
- **Terminology:** FHIR integrates with standardized terminologies such as SNOMED CT, LOINC, and ICD, allowing consistent use of codes and classifications for clinical concepts.
- **Validation:** FHIR provides mechanisms to validate resources against profiles, ensuring that the data conforms to specific rules and constraints, improving data quality and consistency across systems.
- **Interoperability:** FHIR is designed to ensure systems can communicate seamlessly by providing a standardized data format. It promotes interoperability between different healthcare systems, enabling them to share data efficiently.
- **Modularity and Scalability:** FHIR resource can be used individually or in combination, which allows for incremental adoption. This flexibility ensures that FHIR can scale from simple to complex healthcare systems.
- **Exchange Paradigm:** FHIR exchange paradigm refers to the methods used for sharing healthcare data between systems, leveraging modern web standards like RESTful APIs, messaging, and documents. It enables real-time, secure, and scalable exchange of modular data units called resources. This approach facilitates seamless interoperability between healthcare applications, improving data accessibility and patient care.

RESOURCE

FHIR resource is a fundamental component of this standard, representing a specific type of healthcare information in a structured and standardized way. Each resource representing a distinct type of healthcare data element, such as patient information, medications, or observations. These resources are structured entities with defined attributes and properties, which ensure that the data is captured consistently across different systems. Each resource also includes relationships with other resources, allowing them to interact and form a comprehensive representation of a healthcare process. These resources are modular and can be combined or extended to suit specific clinical workflows, enabling flexibility in how they are applied in various healthcare settings while maintaining standardization and interoperability across systems.

Some important resource categories include:

- Clinical Resources: Allergy, Problem, Procedure.
- Administrative Resources: Practitioner, CareTeam, Device, Organization.
- Financial Resources: Claim, Coverage, PaymentNotice.

8.1.2 Resource Content

Structure	UML	XML	JSON	Turtle	R3 Diff	All
Structure						
Name	Flags	Card.	Type	Description & Constraints		
Patient			DomainResource	Information about an individual or animal receiving health care services Elements defined in Ancestors: id, meta, implicitRules, language, text, contained, extension, modifierExtension		
identifier		0..*	Identifier	An identifier for this patient		
active		0..1	boolean	Whether this patient's record is in active use		
name		0..*	HumanName	A name associated with the patient		
telecom		0..*	ContactPoint	A contact detail for the individual		
gender		0..1	code	male female other unknown AdministrativeGender (Required)		
birthDate		0..1	date	The date of birth for the individual		
deceased[x]		0..1		Indicates if the individual is deceased or not		
deceasedBoolean			boolean			
deceasedDateTime			dateTime			
address		0..*	Address	An address for the individual		
maritalStatus		0..1	CodeableConcept	Marital (civil) status of a patient MaritalStatus (Extensible)		
multipleBirth[x]		0..1		Whether patient is part of a multiple birth		
multipleBirthBoolean			boolean			
multipleBirthInteger			integer			
photo		0..*	Attachment	Image of the patient		
contact		0..*	BackboneElement	A contact party (e.g. guardian, partner, friend) for the patient + Rule: SHALL at least contain a contact's details or a reference to an organization		
relationship		0..*	CodeableConcept	The kind of relationship Patient Contact Relationship (Extensible)		
name		0..1	HumanName	A name associated with the contact person		
telecom		0..*	ContactPoint	A contact detail for the person		
address		0..1	Address	Address for the contact person		
gender		0..1	code	male female other unknown AdministrativeGender (Required)		
organization		0..1	Reference(Organization)	Organization that is associated with the contact		
period		0..1	Period	The period during which this contact person or organization is valid to be contacted relating to this patient		
communication		0..*	BackboneElement	A language which may be used to communicate with the patient about his or her health		
language		1..1	CodeableConcept	The language which can be used to communicate with the patient about his or her health Common Languages (Preferred but limited to AllLanguages)		
preferred		0..1	boolean	Language preference indicator		
generalPractitioner		0..*	Reference(Organization Practitioner PractitionerRole)	Patient's nominated primary care provider		
managingOrganization		0..1	Reference(Organization)	Organization that is the custodian of the patient record		
link		0..*	BackboneElement	Link to another patient resource that concerns the same actual person		
other		1..1	Reference(Patient RelatedPerson)	The other patient or related person resource that the link refers to		
type		1..1	code	replaced-by replaces refer seealso LinkType (Required)		

For a full list of resources, refer: [Resource list - FHIR v4.0.1 \(hl7.org\)](https://hl7.org/fhir/2023/04/resource-list/)

Key Part of Resource

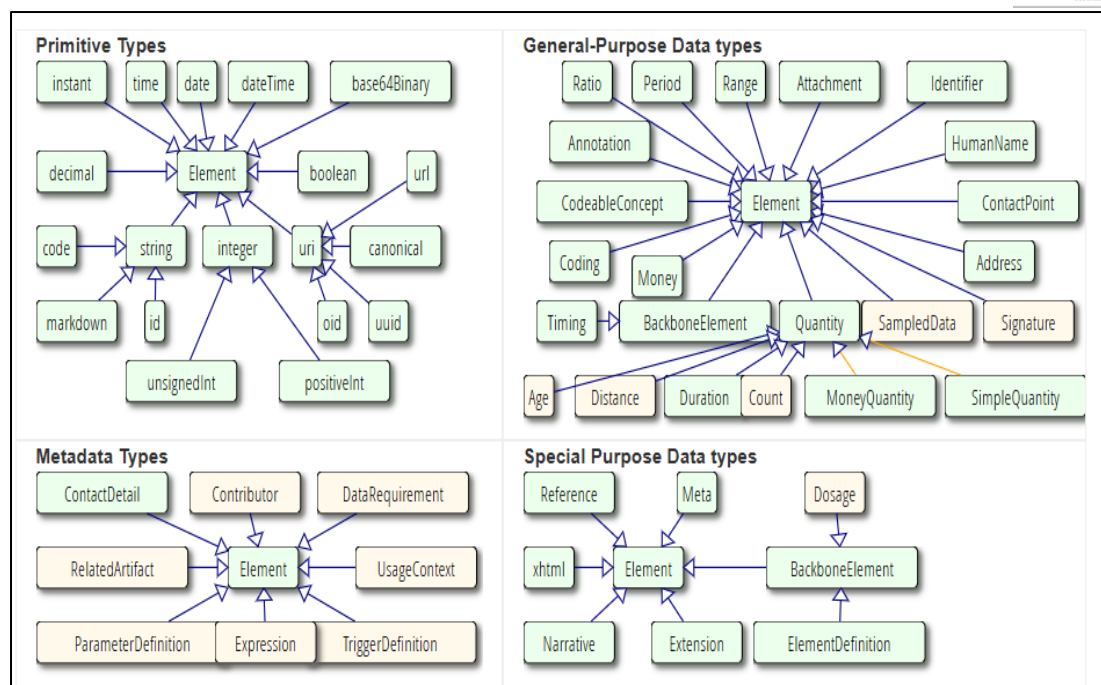
<pre> <Patient xmlns="http://hl7.org/fhir"> <id value="example-01"/> <meta> <versionId value="1"/> <lastUpdated value="2020-07-09T14:58:58.181+05:30"/> <profile value="http://nrces.in/ndhm/fhir/r4/StructureDefinition/Patient"/> </meta> <text> <status value="generated"/> <div xmlns="http://www.w3.org/1999/xhtml">Patient name - ABC, Age- 41 year,Gender- Male</div> </text> <extension url="https://nrces.in/ndhm/fhir/r4/StructureDefinition/someExtension"> <valueCode value="someValue"/> </extension> <identifier> <type> <coding> <system value="http://terminology.hl7.org/CodeSystem/v2-0203"/> <code value="MR"/> <display value="Medical record number"/> </coding> </type> <system value="https://healthid.ndhm.gov.in"/> <value value="22-7225-4829-5255"/> </identifier> <name> <text value="ABC"/> </name> <telecom> <system value="phone"/> <value value="+919818512600"/> <use value="home"/> </telecom> <gender value="male"/> <birthDate value="1981-01-12"/> </Patient> </pre>	Resource Identity & Metadata Human Readable Text Extension with URL to Definition Standard & Structured Data: • Identifier • Name • Telecom • Gender • DOB
---	--

DATA TYPES

Datatypes define the structure and nature of data elements within resources, ensuring consistent representation of information across healthcare systems. They specify how data fields are formatted, such as strings, numbers, or more complex structures. This standardization is crucial for maintaining data integrity and enabling seamless data exchange between different healthcare applications and platforms. By using predefined datatypes, FHIR ensures that both simple and complex data elements are accurately captured and interpreted, enhancing interoperability and data consistency across systems.

Key datatypes:

- **Simple/Primitive Types:** Basic data types with a single, indivisible value like Boolean, integer, string, or date.
- **General-purpose Complex Types:** Reusable clusters of elements that represent structured data like Address, HumanName, or Identifier.
- **Metadata Types:** Used to describe metadata associated with resources.
- **Special Purpose Data Types:** Types created for specific healthcare-related use cases, such as Dosage, Reference, Meta.



Refer: [Datatypes - FHIR v4.0.1 \(hl7.org\)](https://hl7.org/fhir/v4.0.1/datatypes/)

FHIR PARADIGM

FHIR is designed to support a variety of paradigms or approaches to healthcare data exchange, enabling flexible and interoperable communication between systems. It combines different data exchange methods such as **RESTful APIs**, **documents**, **messages**, and **services** to accommodate various workflows and requirements in healthcare settings.

- **RESTful API:** FHIR's most widely used paradigm, where data is exchanged using standard HTTP operations (GET, POST, PUT, DELETE), making it efficient and easy to implement.
- **Documents:** In scenarios where a complete, self-contained set of resources needs to be transmitted, FHIR supports the use of structured documents (e.g., discharge summaries).
- **Messages:** FHIR also supports messaging paradigms, enabling systems to exchange event-driven data, such as lab results or admission notifications.
- **Services:** For more complex interactions, FHIR allows for service-oriented exchanges, and supporting workflows like decision support or scheduling.

These paradigms offer flexibility in how healthcare data is shared and managed, making FHIR adaptable to a wide range of clinical, administrative, and regulatory use cases.

Refer: [Exchange module - FHIR v4.0.1 \(hl7.org\)](https://hl7.org/fhir/v4.0.1/exchange/)

BUNDLE

A FHIR Bundle is a structured container that holds a collection of related resources. It is used for grouping multiple healthcare resources, making it easier to transport them as a single unit. Bundles are commonly used in healthcare data exchange, allowing different resources, like patient records or clinical documents, to be transmitted together.

Use of Bundle with 'type' as 'Document' and 'Collection'

In ABDM, Health Information Types (HI Types/Clinical Artefacts) that represent discrete documents essential for continuity of care are defined using FHIR Bundle with 'type' as *'Document'*. However, the information needed for various claim processing workflows is represented using FHIR Bundle with the 'type' as *'Collection'*.

Use of Bundle with 'type' as 'Document'

The FHIR Bundle type *'Document'* is used to represent clinical artefacts that are discrete and self-contained documents, crucial for maintaining continuity of care in the healthcare system.

How it works:

- A **Bundle** contains a **Composition** resource as its first entry. This Composition serves as the root or header, summarizing the overall document structure.
- The other resources within the Bundle are **referenced by the Composition**, such as Patient, Practitioner, Observation, etc.
- The Bundle ensures that the document, along with its related resources, is exchanged as a cohesive unit.

Example Use Case:

A diagnostic report that includes the test results, interpretation and conclusion. This information is structured using various FHIR resources (e.g. Patient, Diagnostic Report, Observation) and grouped within a Bundle as a document.

Use of Bundle with 'type' as 'Collection'

The FHIR Bundle type *'Collection'* is used to represent sets of related resources into a single package for ease of distribution. A *'Collection'* Bundle functions to organize resources relevant to a specific workflow.

Example Use Case:

A healthcare claim submission that includes details about the patient, services provided, diagnosis, and associated costs. This information is structured using various FHIR resources (e.g., Claim, Patient, Coverage,

Practitioner, Procedure, Condition) and grouped within a Bundle of type 'collection' to represent the complete claim request.

Key Elements of a FHIR Bundle resource

- **Bundle.type:** Defines the purpose of the bundle, indicating how it should be processed (e.g., 'document', 'collection'). This helps categorize the bundle's role in a healthcare workflow.
- **Bundle.timestamp:** The exact date and time when the bundle was created. It ensures accurate tracking of when the information within the bundle was assembled.
- **Bundle.identifier:** A unique value that distinguishes a specific Bundle from others, ensuring it can be identified across systems. It plays a key role in maintaining data integrity and traceability in healthcare exchanges.
- **Bundle.entry:** Each **entry** in a Bundle represents an individual resource that is part of the overall collection. A Bundle may contain one or more entries, depending on how many resources are being grouped. Each entry consists of several components:
 - **Full URL:** A reference to the specific resource, often providing a resolvable URL where the resource can be accessed.
 - **Resource:** The actual FHIR resource (e.g., Patient, Observation, or Encounter) that is being included in the Bundle.

Refer: [Bundle - FHIR v4.0.1 \(hl7.org\)](http://hl7.org/fhir/v4.0.1/bundle)

FHIR IMPLEMENTATION GUIDE

FHIR Implementation Guide (IG) is a document that provides specific guidelines on how to implement the HL7 Fast Healthcare Interoperability Resources (FHIR) standard in a particular healthcare context or for a specific use case. It describes how the standard should be applied, customized, or extended for particular needs, ensuring interoperability and consistency in implementations across different systems.

Typically, a FHIR Implementation Guide contains:

- **Profiles:** Customized versions of standard FHIR resources that specify constraints, extensions, and usage guidelines for the particular context.
- **Extensions:** Custom fields or data points added to FHIR resources that aren't covered by the base standard.
- **ValueSets:** Defined sets of allowable codes or terminologies (such as ICD, SNOMED CT) that can be used in certain elements of FHIR resources.
- **Examples:** Sample FHIR resource instances to demonstrate correct usage.

- **Narratives and Guidance:** Detailed explanations about how and why certain decisions were made, and how the FHIR resources should be used together.

By following an IG, healthcare organizations ensure that their systems are capable of exchanging information in a standardized manner, promoting interoperability.

FHIR Implementation Guide for Ayushman Bharat Digital Mission

The FHIR Implementation Guide for ABDM is built on FHIR Version R4 (4.0.1), it establishes the minimum conformance requirements for accessing health data to ensure continuity of care in ABDM. This guide defines the essential health record artifacts to be captured and exchanged in line with the ABDM.

It references key standards and coding systems from the National Digital Health Blueprint (NDHB), EHR Standards for India (2016), and regulatory bodies like the Medical Council of India (MCI), Pharmacy Council of India (PCI), and Health Claim Exchange Platform (NHCX).

Refer: [Home - FHIR Implementation Guide for ABDM](#)

Reading FHIR Profiles

The ABDM FHIR Implementation Guide includes several profiles to capture and exchange health data. Understanding how to read and interpret these profiles is essential for developers and healthcare providers. Below are key concepts to help understand ABDM FHIR profiles:

1. Statistics/References

- Provides a **human-readable summary** of changes made to the base FHIR resources. It refers to the **Differential View**, showing which elements have been modified, constrained, or extended in the profile.

2. Differential View

- The **Differential View** lists the specific changes applied to the FHIR resource while creating a profile. This includes constraints, extensions, and customizations tailored for ABDM.
- Only the modified elements are displayed, making it easier for implementers to focus on what has been changed from the standard FHIR resource.

3. Mandatory Element

- Elements with cardinality '**1..1**' or '**1..***' are mandatory and must always be present in the resource. These elements are critical for the proper functioning of the data exchange and cannot be omitted.

4. Must Support S

- **‘MUST Support’** elements are optional to include but must be supported by receiving systems. The **Healthcare Information User (HIU)** must be able to process these elements if present, while the **Healthcare Information Provider (HIP)** can choose whether to include them. This ensures that systems can handle optional data when available.

Refer: [Formats - FHIR v4.0.1 \(hl7.org\)](https://hl7.org/fhir/v4.0.1/formats.html)

Differential Table
Key Elements Table
Snapshot Table
Statistics/References
All

This structure is derived from [DiagnosticReport](#)

Summary

Mandatory: 3 elements (11 nested mandatory elements)
 Must-Support: 16 elements
 Fixed Value: 3 elements

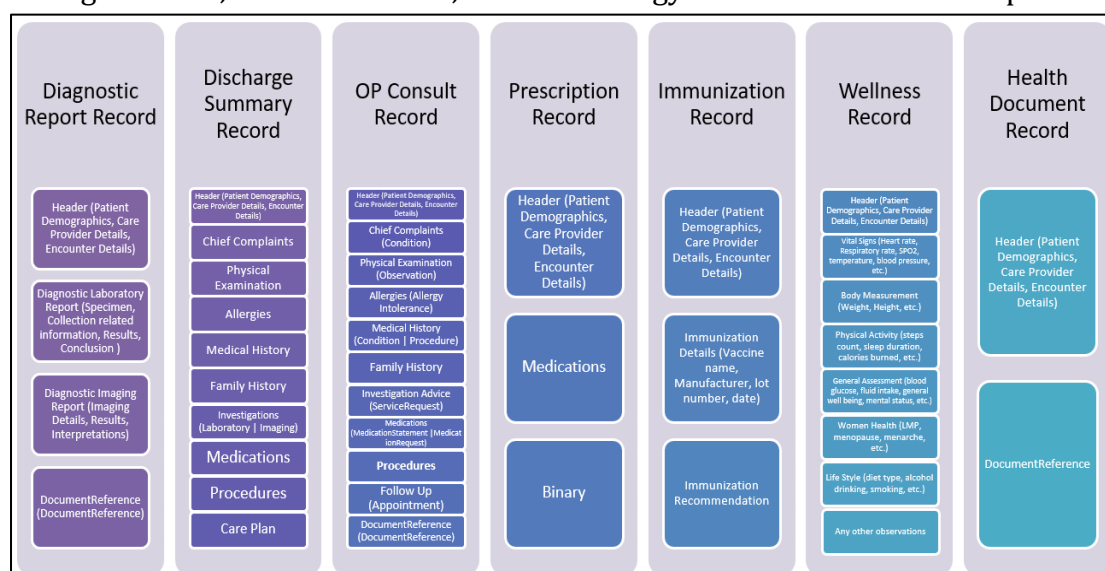
Structures

This structure refers to these other structures:

- [MedicationRequest](#) (<https://nrces.in/ndhm/fhir/r4/StructureDefinition/MedicationRequest>)
- [ServiceRequest](#) (<https://nrces.in/ndhm/fhir/r4/StructureDefinition/ServiceRequest>)
- [CarePlan](#) (<https://nrces.in/ndhm/fhir/r4/StructureDefinition/CarePlan>)
- [ImmunizationRecommendation](#) (<https://nrces.in/ndhm/fhir/r4/StructureDefinition/ImmunizationRecommendation>)
- [Patient](#) (<https://nrces.in/ndhm/fhir/r4/StructureDefinition/Patient>)
- [Encounter](#) (<https://nrces.in/ndhm/fhir/r4/StructureDefinition/Encounter>)
- [Organization](#) (<https://nrces.in/ndhm/fhir/r4/StructureDefinition/Organization>)

FHIR Profiles for ABDM

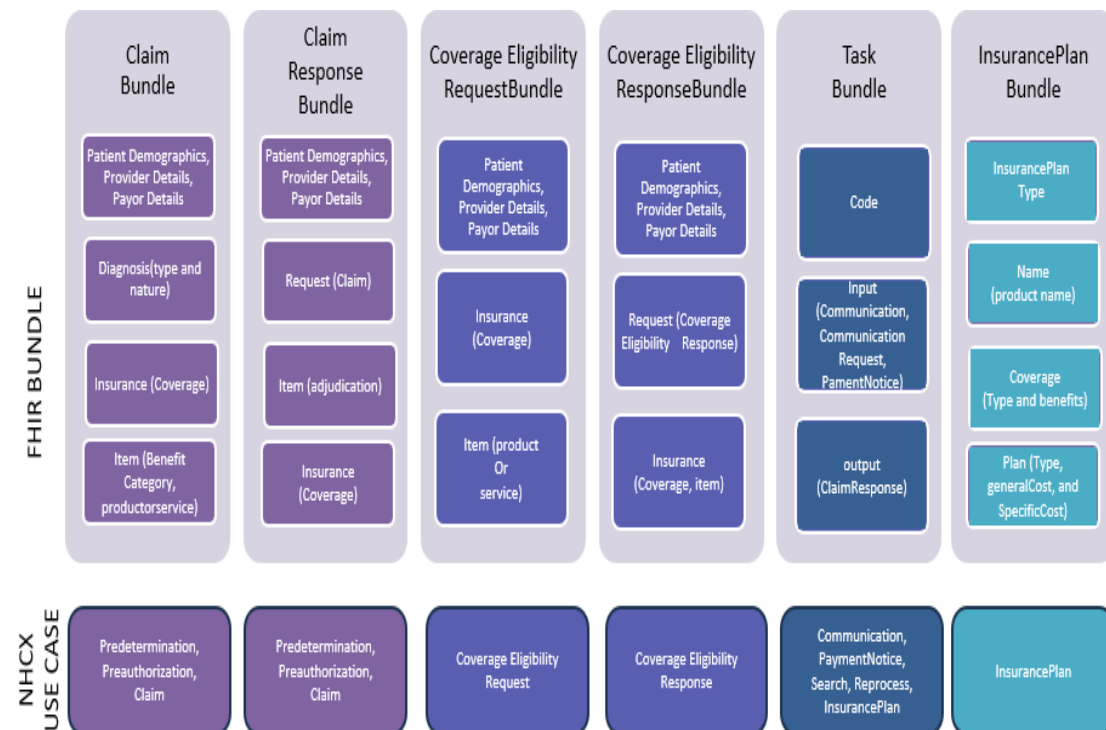
The ABDM artifacts aim to cover a wide range of health record document sharing within care settings. These artifacts ensure comprehensive data capture and exchange to support continuity of care. This includes 07 Clinical Artifacts, 01 Billing Artifacts, 38 Core Profiles, 42 Terminology ValueSets and 92 examples.



Refer: [ABDM Profiles - FHIR Implementation Guide for ABDM](#)

FHIR Profiles for NHCX

The NHCX artifacts are designed to facilitate standardized and efficient exchange of health claim-related information among payers, providers, beneficiaries, and other stakeholders. These artifacts support a range of processes related to health claims, including eligibility checks, pre-authorization requests, claims submissions, and payment notifications. They ensure that data is exchanged in an interoperable, machine-readable, and auditable format, promoting accurate and timely processing of health claims. This includes 06 Health Claim Artifacts, 11 Core Profiles, 10 Codesystem 17 Valuesets, and 51 examples.



Refer: [NHCX Profiles - FHIR Implementation Guide for ABDM](#)

IMPLEMENTING AND VALIDATING FHIR PROGRAMMATICALLY IN JAVA

This document provides a comprehensive guide for creating and validating FHIR resources programmatically using Java. We will use the **HAPI FHIR library**, a popular choice for working with FHIR in Java.

HAPI FHIR Library

The **HAPI FHIR library** is an open-source Java framework that simplifies the process of working with FHIR (Fast Healthcare Interoperability Resources) resources. It provides comprehensive support for creating, manipulating, validating, serializing, and parsing FHIR resources.

Refer: [HAPI FHIR - The Open Source FHIR API for Java](#)

Key Features of HAPI FHIR Library

- **Complete FHIR Support:** HAPI FHIR offers full support for all FHIR resource types. This includes creating, validating, serializing, and parsing each resource, ensuring compliance with FHIR standards.
- **Built-in Validators:** The library includes a robust validation framework that checks FHIR resources against the FHIR specification. You can validate resources using predefined profiles, custom profiles, or specific rules.
- **Serialization & Parsing:** The library provides seamless serialization and deserialization capabilities for FHIR resources in both **JSON** and **XML** formats. The parsing functions convert the structured data into Java objects for easy manipulation.
- **Extensive Resource Coverage:** The library contains Java classes representing every resource in the FHIR specification, such as Patient, Practitioner

HAPI FHIR Dependencies

- **hapi-fhir-structures-r4:** This library provides Java classes and structures for all FHIR R4(4.0.1) resources, enabling the creation and manipulation of FHIR resources in Java applications.
- **hapi-fhir-validation:** This module performs validation of FHIR resources against standard and custom FHIR profiles, ensuring that the resources conform to the specified FHIR rules and constraints.
- **hapi-fhir-validation-resources-r4:** This library includes predefined resources and profiles required for validating specific FHIR R4 ensuring proper compliance with FHIR.

Creating FHIR Resources Programmatically

Prerequisites

Before you start, ensure you have:

- **Java Development Kit (JDK):** Ensure JDK 11 or higher is installed.
- **Integrated Development Environment (IDE):** Use an IDE like IntelliJ IDEA or Eclipse.

Setting Up the Development Environment

✓ Add HAPI FHIR Dependency

Add the HAPI FHIR dependency to your '*pom.xml*' file if you are using Maven

```
<!-- https://mvnrepository.com/artifact/ca.uhn.hapi.fhir/hapi-fhir-structures-r4 -->
<dependency>
  <groupId>ca.uhn.hapi.fhir</groupId>
  <artifactId>hapi-fhir-structures-r4</artifactId>
  <version>6.4.3</version>
</dependency>

<!-- https://mvnrepository.com/artifact/ca.uhn.hapi.fhir/hapi-fhir-validation -->
<dependency>
  <groupId>ca.uhn.hapi.fhir</groupId>
  <artifactId>hapi-fhir-validation</artifactId>
  <version>6.4.3</version>
</dependency>

<!-- https://mvnrepository.com/artifact/ca.uhn.hapi.fhir/hapi-fhir-validation-resources-r4 -->
<dependency>
  <groupId>ca.uhn.hapi.fhir</groupId>
  <artifactId>hapi-fhir-validation-resources-r4</artifactId>
  <version>6.4.3</version>
</dependency>
```

Alternatively, if using Gradle, add the following to your build.gradle:

```
// https://mvnrepository.com/artifact/ca.uhn.hapi.fhir/hapi-fhir-structures-r4
implementation group: 'ca.uhn.hapi.fhir', name: 'hapi-fhir-structures-r4', version: '6.4.3'

// https://mvnrepository.com/artifact/ca.uhn.hapi.fhir/hapi-fhir-validation
implementation group: 'ca.uhn.hapi.fhir', name: 'hapi-fhir-validation', version: '6.4.3'

// https://mvnrepository.com/artifact/ca.uhn.hapi.fhir/hapi-fhir-validation-resources-r4
implementation group: 'ca.uhn.hapi.fhir', name: 'hapi-fhir-validation-resources-r4', version: '6.4.3'
```

✓ Set Up The Project

Create a new Java project in your IDE and configure it to include the HAPI

Creating Resource

To create a FHIR resource:

1. **Define the Resource:** Instantiate a FHIR resource object (e.g., Bundle, Composition, Patient).
2. **Populate the Resource:** Set attributes and values according to the FHIR specification.
3. **Serialize the Resource:** Convert the resource to JSON or XML format.

Example: Creating a Patient Resource

<pre> public static Patient populatePatientResource() { Patient patient = new Patient(); patient.setId("1efe03bf-9506-40ba-bc9a-80b0d5045afe"); patient.getMeta(). setVersionId("1"). setLastUpdatedElement(new InstantType("2020-07-09T14:58:58.181+05:30")) .addProfile("https://nrces.in/ndhm/fhir/r4/StructureDefinition/Patient"); patient.addIdentifier() .setType(new CodeableConcept(new Coding("http://terminology.hl7.org/CodeSystem/v2-0203", "MR", "Medical record number"))) .setSystem("https://healthid.ndhm.gov.in") .setValue("22-7225-4829-5255"); patient.addName(). setText("ABC"); patient.addTelecom(). setSystem(ContactPointSystem.PHONE) .setValue("+919818512600") .setUse(ContactPointUse.HOME); patient.setGender(AdministrativeGender.MALE) .setBirthDateElement(new DateType("1981-01-12")); return patient; } </pre>	<pre> { "resourceType": "Patient", "id": "1efe03bf-9506-40ba-bc9a-80b0d5045afe", "meta": { "versionId": "1", "lastUpdated": "2020-07-09T14:58:58.181+05:30", "profile": ["https://nrces.in/ndhm/fhir/r4/StructureDefinition/Patient"] }, "identifier": [{ "type": { "coding": [{ "system": "http://terminology.hl7.org/CodeSystem/v2-0203", "code": "MR", "display": "Medical record number" }] }, "system": "https://healthid.ndhm.gov.in", "value": "22-7225-4829-5255" }], "name": [{ "text": "ABC" }], "telecom": [{ "system": "phone", "value": "+919818512600", "use": "home" }], "gender": "male", "birthDate": "1981-01-12" } </pre>
Patient Resource in Java	Patient Resource in JSON

VALIDATING FHIR RESOURCES

Validation Tools

1. **HAPI FHIR Validator:** A built-in validation tool available in the HAPI FHIR library.
 - URL: [HAPI FHIR Validator](#)
2. **FHIR Validator CLI:** Command-line interface for FHIR validation.
 - URL: [FHIR Validator CLI](#)
3. **FHIR GUI Validator:**
 - URL: [FHIR GUI Validator](#)

Validation Aspects

When validating a FHIR resource, several key aspects are checked to ensure compliance with the FHIR specification:

1. **Structure:** Verifies that the resource conforms to the FHIR specification, with no extra or undefined elements present.
2. **Cardinality:** Ensures that properties adhere to their defined cardinality (minimum and maximum occurrences).
3. **Value Domains:** Confirms that property values match their data types and enumerated codes are valid.
4. **Coding/CodeableConcept Bindings:** Validates that Coding or CodeableConcept elements use correct system URL, codes and display values as per required valuesets.
5. **Invariants:** Checks that all constraints or co-occurrence rules are satisfied (e.g., if one field is present, another must also be present).
6. **Profiles:** Ensures compliance with any specific rules defined in FHIR profiles, CapabilityStatements, ImplementationGuides, or other contexts.
7. **Business Rules:** Includes additional checks like duplicate detection, reference resolution, and authorization validation.

These aspects ensure that FHIR resources are structurally sound and meet both clinical and business requirements.

Validation Process

1. **HAPI FHIR Validator:** To validate a resource using the HAPI FHIR library, following are the steps:

- **Step 1: Load NPM Package**
 - Download the [package.tgz](#) containing all the Structure Definitions, CodeSystems, and ValueSets from ABDM FHIR Implementation Guide.
 - Add package.tgz to the class path ("*src/main/resource*")
 - "**package.tgz**" is essential for validating resources against ABDM and NHCX profiles and terminologies.

```
// Create NpmPackageValidationSupport instance
NpmPackageValidationSupport npmValidationSupport = new NpmPackageValidationSupport(ctx);

// Load package from classpath
npmValidationSupport.loadPackageFromClasspath("classpath:<Name of the package>");
```

For more Information Refer: [Instance Validator using package](#)

- **Step 2: Setup a validation support chain**
 - Establish a validation support chain that incorporates the core FHIR structure definitions. This chain includes FHIR StructureDefinition and FHIR's built-in vocabulary (such as ValueSet and CodeSystem resources).
 - It involves an in-memory terminology service, module caching, and support for validating codes with CodeSystems that are not distributed as part of the FHIR specification.

```
// Create a ValidationSupportChain
ValidationSupportChain validationSupportChain = new ValidationSupportChain(
    npmValidationSupport,           // NPM package support
    new DefaultProfileValidationSupport(ctx), // Default profile validation
    new InMemoryTerminologyServerValidationSupport(ctx), // Terminology validation
    new CommonCodeSystemsTerminologyService(ctx), // Common code systems
    new SnapshotGeneratingValidationSupport(ctx) // Generate snapshots of StructureDefinitions
);
// Add Caching layer on top of the validation chain
CachingValidationSupport validationSupport = new CachingValidationSupport(validationSupportChain);
```

Refer: [Validation Support Modules](#)

- **Step 3: Register validator and validate resource**

```
// Initialize FHIR context for R4
FhirContext ctx = FhirContext.forR4();

// Declare FhirValidator and FhirInstanceValidator
FhirValidator validator;
FhirInstanceValidator fhirInstanceValidator;

// Initialize the validator
validator = ctx.newValidator();

// Set up FhirInstanceValidator with the validation support chain
fhirInstanceValidator = new FhirInstanceValidator(validationSupport);
validator.registerValidatorModule(fhirInstanceValidator);

// Validate a FHIR resource
ValidationResult outcome = validator.validateWithResult(resource);

// Print the overall validation outcome
System.out.println(outcome);

// Loop through and print individual validation messages
for (SingleValidationMessage next : outcome.getMessages()) {
    System.out.println(next.getSeverity() + " - " + next.getLocationString() + " - " + next.getMessage());
}
```

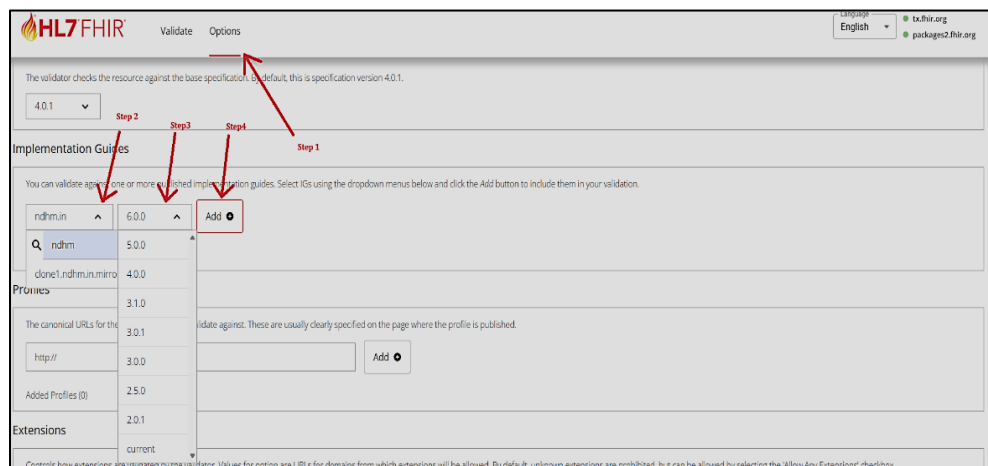
2. **FHIR Validator CLI:** To validate a resource using the JAR file provided by HL7, use following command:

"Java -jar <path to validator_cli.jar> <file_name> -ig ndhm.in#<ig-version>"

For Documentation refer: [Using the FHIR Validator - FHIR - Confluence \(hl7.org\)](https://confluence.hl7.org/pages/viewpage.action?pageId=14111111)

3. FHIR GUI Validator

- Navigate to <https://validator.fhir.org>
- Add Implementation Guide



The screenshot shows the HL7 FHIR Validator interface. The 'Options' tab is selected. Under 'Implementation Guides', the 'ndhm.in' guide is selected with version '6.0.0'. The 'Add' button is highlighted. Other sections like 'Profiles' and 'Extensions' are also visible.

- **Step 1:** Click on the "Options" tab
- **Step 2:** Select the implementation guide "ndhm.in"
- **Step 3:** Choose the latest version from the dropdown.
- **Step 4:** Click on the "Add" button to add IG.
- Validation process



The screenshot shows the 'Validate' tab of the HL7 FHIR Validator. The 'ENTER RESOURCE' section is active. At the bottom left, the 'Validate' button is highlighted. The status bar at the bottom indicates 'Common Validation Options...' and 'FHIR Resource (34)'.

- **Step 1:** Paste the FHIR resource that need to be validated.
- **Step 2:** Click on the "Validate" button to begin the validation process. The validator will check the resource against the selected implementation guide and provide feedback on any errors or warnings.

IMPLEMENTATION REFERENCE

- **Implementation Guide**
 - HL7 : [Index - FHIR v4.0.1 \(hl7.org\)](#)
 - ABDM : [Home - FHIR Implementation Guide for ABDM](#)
- **Implementation Libraries**
 - Java : [HAPI FHIR - The Open Source FHIR API for Java](#)
 - C# : [Firely .NET SDK | The official .NET SDK for HL7 FHIR](#)
 - JavaScript : [fhir-kit-models-npm \(npmjs.com\)](#)
 - Additional : [Open Source Implementations - FHIR - Confluence](#)
- **Tool**
 - Validator cli : [Using the FHIR Validator - FHIR - Confluence](#)
- **Schema**
 - JSON : [JSON Schema](#)
- **Usage Sample**
 - Java : [Usage Sample code – JAVA](#)
 - .NET : [Usage Sample code - DOTNET](#)

ANNEXTURE

S.N.	Use Case	API End Point	Flow	FHIR Bundle
1	Coverage Eligibility	/coverageeligibility/check	provider->NHCX->payer	CoverageEligibilityRequestBundle
2	Coverage Eligibility	/coverageeligibility/on_check	payer->NHCX->provider	CoverageEligibilityResponseBundle
3	Preauthorization	/preauth/submit	provider->NHCX->payer	ClaimBundle
4	Preauthorization	/preauth/on_submit	payer->NHCX->provider	ClaimResponseBundle
5	Predetermination	/predetermination/submit	provider->NHCX->payer	ClaimBundle
6	Predetermination	/predetermination/on_submit	payer->NHCX->provider	ClaimResponseBundle
7	Claim	/claim/submit	provider->NHCX->payer	ClaimBundle
8	Claim	/claim/on_submit	payer->NHCX->provider	ClaimResponseBundle
9	Request Additional Attachments	/communication/request	payer->NHCX->provider	TaskBundle
10	Send Attachments	/communication/on_request	provider->NHCX->payer	TaskBundle
11	Payment	/paymentnotice/request	payer->NHCX->provider	TaskBundle
12	Payment	/paymentnotice/on_request	provider->NHCX->payer	TaskBundle
13	Status Check	/hcx/status	provider->NHCX,Payer->NHCX	NA
14	Status Check	/NHCX/on_status	provider->NHCX,Payer->NHCX	NA
15	Reprocess	/task/submit	provider->NHCX->payer	TaskBundle
16	Reprocess	/task/on_submit	payer->NHCX->provider	TaskBundle
17	Search	/search/submit	NHA->NHCX->Payer	TaskBundle
18	Search	/search/on_submit	payer->NHCX->NHA	TaskBundle